

Indice

1	Introduzione all'IA e Agenti Intelligenti	1
1.1	Cos'è l'IA	1
2	Ricerca nello spazio degli stati	2
2.1	Ricerca non informata (blind)	2
2.2	Ricerca informata (euristica)	3
3	Ricerca con avversario (giochi)	3
4	Constraint Satisfaction Problems (CSP)	3
5	Logica proposizionale e rappresentazione della conoscenza	4
6	Logica del primo ordine (FOL)	4
7	Ontologie e Pianificazione	5
7.1	Tassonomie/Ontologie	5
7.2	Pianificazione	6
8	Machine Learning	6
8.1	Classificazione	6
8.2	Alberi di decisione	6
8.3	Reti neurali	6

1 Introduzione all'IA e Agenti Intelligenti

1.1 Cos'è l'IA

IA: intelligenza (facoltà di pensare, adattarsi, imparare) ottenuta artificialmente. **Automazione** $\neq$ **intelligenza**: eseguire istruzioni fisse non basta, serve adattarsi a situazioni nuove.

- **Test di Turing** (1950): un umano dialoga alla cieca con umano+macchina; se non distingue chi è chi, la macchina “passa”. Critica: stesso output non implica stessa comprensione (es. chi risponde “Piemonte” sapendo geografia vs. tirando a caso).
- **Stanza Cinese** (Searle, 1980): manipolare simboli sintatticamente (manuale di regole) non implica comprensione semantica — critica alla Strong AI.
- **Strong AI** (riprodurre davvero il pensiero umano) vs **Weak AI** (task-oriented, basta *comportamento razionale*: approccio del corso, R&N).

Agente = sistema che percepisce l'ambiente (sensori) e agisce su di esso (attuatori); agente+ambiente = binomio inscindibile. **Sequenza percettiva** = storia di tutte le percezioni ricevute.

Razionalità: un agente razionale sceglie, per ogni sequenza percettiva, l'azione che massimizza la prestazione *attesa* (non quella reale/onnisciente) data la sua conoscenza. Dipende da: azioni disponibili, misura di prestazione, conoscenza dell'ambiente, sequenza percettiva. Razionale $\neq$ di successo, $\neq$ onnisciente. La percezione stessa può essere un'azione (es. guardare prima di attraversare).

PEAS: Performance measure, Environment, Actuators, Sensors — si specifica sempre progettando un agente.

Proprietà dell'ambiente

Asse	Facile	Difficile
Osservabilità	completa	parziale
Determinismo	deterministico	stocastico (strategico se dipende da altri agenti)
Episodicità	episodico	sequenziale
Dinamicità	statico	dinamico
Discretezza	discreto	continuo
N. agenti	singolo	multiagente

Parziale osservabilità spesso *appare* come stocasticità. Caso più difficile: parzialmente osservabile, stocastico, sequenziale, dinamico, continuo, multiagente.

Agente = architettura + programma. Funzione agente (ideale, storia intera $\rightarrow$ azione) vs programma agente (percezione corrente + stato interno $\rightarrow$ azione).

Tipologie di agente (sostituzione crescente)

1. **Reattivo semplice:** regole condizione-azione sulla sola percezione corrente; funziona solo se ambiente completamente osservabile; rischio loop infiniti (serve randomizzazione).
2. **Reattivo basato su modello:** mantiene stato interno + modello di come evolve il mondo $\rightarrow$ gestisce osservabilità parziale.
3. **Basato su obiettivi:** sceglie azioni per raggiungere un goal (ragionamento ipotetico/pianificazione); basta cambiare il goal per cambiare comportamento.
4. **Basato sull'utilità:** funzione di utilità per scegliere la *migliore* fra più soluzioni (non solo una qualsiasi).
5. **Che apprende:** aggiunge Critico (valuta prestazione) + Modulo di apprendimento + Generatore di problemi (azioni esplorative).

Paradigma dichiarativo vs procedurale: dichiarativo separa *cosa* (KB) da *come* (motore generale di inferenza/ricerca), riusabile su problemi diversi — vs imperativo (sequenza di passi ad hoc). Esempio guida: **mondo dei blocchi** — l'agente non esegue un piano scritto dal programmatore, lo *costruisce* lui stesso cercando nello spazio degli stati.

2 Ricerca nello spazio degli stati

Problema di ricerca = 4-tupla: stato iniziale, funzione successore (azioni+transizioni), test obiettivo, costo del cammino. **Spazio degli stati** (astratto) $\neq$ **albero/grafico di ricerca** (struttura dati costruita dall'algoritmo: nodi con genitore, azione, profondità, costo).

Frontiera: nodi generati non ancora espansi. Criteri di valutazione: **completezza** (trova sempre soluzione se esiste), **ottimalità** (trova la soluzione a costo minimo), **complessità temporale/spaziale** (in nodi generati/mantenuti, O -grande). Parametri: b branching factor, d profondità soluzione più superficiale, m profondità massima.

2.1 Ricerca non informata (blind)

Strategia	Frontiera	Completa	Ottima	Tempo	Spazio
Ampiezza (BFS)	FIFO	Sì (b finito)	Sì (costi unif.)	$O(b^{d+1})$	$O(b^{d+1})$
Costo uniforme	priorità su $g(n)$	Sì ($c > 0$)	Sì ($c > 0$)	$O(b^{1+\lceil C^*/\epsilon \rceil})$	idem
Profondità (DFS)	LIFO/stack	No [†]	No	$O(b^m)$	$O(bm)$ [$O(m)$ backtr.]
Prof. limitata (l)	stack+limite	No se $l < d$	No	$O(b^l)$	$O(bl)$
Iterative deepening	stack ricostr.	Sì	Sì	$O(b^d)$	$O(bd)$
Bidirezionale	doppia frontiera	Sì	Sì (costi unif.)	$O(b^{d/2})$	$O(b^{d/2})$

[†] completa solo se tutti i cammini sono finiti.

IDDFS è generalmente la scelta migliore quando d non è nota: combina completezza/ottimalità di BFS con spazio $O(bd)$ come DFS. Marcare nodi visitati (albero $\rightarrow$ grafo) evita di riesplorare stati ripetuti.

2.2 Ricerca informata (euristica)

$f(n)$ = funzione di valutazione; $h(n)$ stima il costo residuo a un goal.

- **Greedy**: $f(n) = h(n)$. Veloce ma non completa/ottima (eredita difetti DFS, l'euristica può ingannare).
- **A***: $f(n) = g(n) + h(n)$ — g guarda indietro (esatto), h guarda avanti (stima). **Ottimo se h è ammissibile** ($h(n) \leq h^*(n)$, mai sovrastima) su alberi; su **grafi** serve **consistenza/monotonicità**: $h(n) \leq c(n, a, n') + h(n')$ (garantisce f non decrescente lungo il cammino). Ogni euristica monotona è ammissibile, non viceversa. **A*** è **ottimamente efficiente** (nessun altro algoritmo con la stessa euristica espande meno nodi) ma mantiene tutti i nodi in memoria (come BFS).
- **IDA***, **RBFS**: riducono la memoria a $O(bd)$ (lineare) usando un upper bound dinamico, a costo di lavoro ripetuto.

Effective branching factor b^* : $N + 1 = 1 + b^* + \dots + (b^*)^d$ — più vicino a 1, migliore/più informativa l'euristica.

Dominanza: se $h_2(n) \geq h_1(n) \forall n$ (entrambe ammissibili), h_2 domina $h_1 \Rightarrow A^*$ con h_2 espande un sottoinsieme dei nodi espansi con h_1 .

Problemi rilassati: rimuovendo vincoli si ottiene un supergrafo; il costo della soluzione ottima nel problema rilassato è sempre un'euristica ammissibile. Esempio 8-puzzle: h_1 =tessere fuori posto, h_2 =distanza di Manhattan (h_2 domina h_1). Più euristiche ammissibili non comparabili $\rightarrow h(n) = \max\{h_1(n), \dots, h_k(n)\}$ è ammissibile e dominante su tutte.

3 Ricerca con avversario (giochi)

Ambiente **multi-agente competitivo**: obiettivi in conflitto. Giochi a **informazione completa/perfetta** (stato visibile a tutti, es. scacchi) vs **imperfetta** (es. poker); **deterministici** vs **stocastici** (dadi). Il modulo tratta giochi deterministici, 2 giocatori, **somma zero** ($u(A) = -u(B)$, un solo valore per nodo basta).

Albero di gioco: radice=stato iniziale, nodi MAX/MIN alternati, foglie=utilità terminale. **Ipotesi di pessimismo**: l'avversario (MIN) è infallibile e ottimale.

Minimax: valore ricorsivo bottom-up

$$\text{Minimax}(n) = \begin{cases} \text{Utilità}(n) & n \text{ terminale} \\ \max_{s \in \text{Succ}(n)} \text{Minimax}(s) & n \text{ nodo MAX} \\ \min_{s \in \text{Succ}(n)} \text{Minimax}(s) & n \text{ nodo MIN} \end{cases}$$

Tempo $O(b^m)$, spazio $O(bm)$. **Completo e ottimo** (se entrambi giocano ottimo). Generalizza a n giocatori con vettori di utilità (possibili alleanze).

Teoria delle decisioni: maximax (ottimistico, max dei max), **maximin** (pessimistico/conservativo — logica di Minimax), minimax regret (minimizza il rimpianto massimo = best payoff scenario — payoff reale).

Potatura alfa-beta: α = miglior garanzia per MAX (lower bound, aggiornato nei nodi MAX), β = miglior garanzia per MIN (upper bound, nodi MIN). Si pota quando $\beta \leq \alpha$. Stesso risultato di Minimax, ma nel caso migliore $O(b^{m/2})$ (dimezza l'esponente). Dipende **criticamente dall'ordinamento delle mosse** (killer move per primo): caso medio $O(b^{3m/4})$, caso peggiore = Minimax puro. **Trasposizioni**: stati raggiunti da ordini di mosse diversi $\rightarrow$ hash table per non riesplorarli.

Real-time/cutoff: si taglia a profondità limitata con **funzione di valutazione euristica** (combinazione lineare pesata di feature: $\text{eval}(s) = \sum_i w_i f_i(s)$). Rischio: **problema dell'orizzonte** (l'algoritmo non vede un ribaltamento imminente) $\rightarrow$ mitigato dalla **quiescenza** (Berliner): tagliare solo dove la valutazione è stabile.

Storici: **DeepBlue** (scacchi, iterative deepening + alfa-beta + tabelle trasposizione, 200M pos/sec, batté Kasparov 1996), **AlphaGo** (Go, 2015, reti neurali + ricerca ad albero + self-play).

4 Constraint Satisfaction Problems (CSP)

Definizione: variabili $X_1, \dots, X_n$, domini $D_1, \dots, D_n$, vincoli $C_1, \dots, C_m$. Assegnamento **completo** (tutte le var.) e **consistente** (nessun vincolo violato) = **soluzione**. Vincoli: unari, binari (grafo di vincoli), n-ari (ipergrafo, es. Alldifferent, criptoaritmetica). Vincoli rigidi vs criteri di preferenza (soft).

Come problema di ricerca: stato iniziale = $\{\}$, successore = assegna una variabile, goal = assegnamento completo. **Commutatività**: l'ordine di assegnamento non conta $\rightarrow$ si fissa sempre prima la variabile poi il valore, riducendo il branching da $n \cdot d$ a d per livello (foglie: d^n invece di $n! \cdot d^n$).

Generate-and-test: impraticabile (esplosione combinatoria). **Backtracking** (DFS + potatura sui vincoli): soffre di **thrashing** se non guidato da euristiche/inferenza.

Euristica variabile: **MRV** (Minimum Remaining Values / fail-first: scegli la variabile più vincolata, per fallire il prima possibile) + **degree heuristic** (tie-break: più vincoli con variabili non assegnate). **Euristica valore:** **LCV** (Least Constraining Value: scegli il valore che lascia più opzioni ai vicini). *Non contraddittorio:* variabile = “rischia di fallire subito”, valore = “minimizza il rischio una volta lì”.

Consistenza locale (crescente forza/costo): **Forward checking** (propaga solo ai vicini diretti della variabile appena assegnata, si abbina a MRV) → **Node consistency** (vincoli unari) → **Arc consistency/AC-3** (Mackworth 1977: propaga su tutto il grafo tramite coda di archi, **REVISE** elimina valori senza supporto, $O(n^2 d^3)$; **incompleta:** arc-consistent non implica soluzione, es. triangolo a 2 colori) → **Path consistency/PC-2** (triplette di variabili, più forte). **k-consistency:** fortemente n -consistent → risolvibile senza backtracking, ma verificarlo è esponenziale.

Backtracking cronologico torna sempre alla variabile precedente, anche se non causa del conflitto. **Backjumping:** usa il **conflict set** per saltare alla vera responsabile (se combinato con FC arricchito, è ridondante). **Conflict-directed backjumping:** aggiorna dinamicamente i conflict set, gestisce gli **assegnamenti NOGOOD** (assegnamenti parziali mai estendibili a soluzione, scoperti solo esplorando) — è una forma di apprendimento.

Vincoli globali: **Alldifferent** (pigeonhole: se $< n$ valori disponibili, fallimento immediato), **Atmost**($N, \dots$) (vincoli di risorse). Applicazioni: scheduling, routing, timetabling, Sudoku (tutto Alldifferent).

5 Logica proposizionale e rappresentazione della conoscenza

Agente basato su conoscenza: KB + **tell** (asserisci) + **ask** (interroga); ogni risposta ad ask deve essere conseguenza logica delle tell fatte. Dato (grezzo) → informazione (significato) → conoscenza (relazioni fra informazioni).

Modello m : mondo possibile che fissa il valore di verità delle formule. $M(\alpha)$ = insieme dei modelli di α . **Conseguenza logica** $A \models B$: B vera in *tutti* i modelli in cui A è vera $\iff M(A) \subseteq M(B)$. **Equivalenza** $A \equiv B$: $M(A) = M(B)$. **Validità/tautologia:** vera in ogni modello. **Insoddisfacibilità:** falsa in ogni modello. **Soddisfacibilità:** vera in almeno un modello (P valida $\iff \neg P$ insoddisfacibile).

Inferenza: processo **sintattico**. $KB \vdash_i A$: A derivabile da KB con l'algoritmo i . **Correttezza** (soundness): $KB \vdash_i A \Rightarrow KB \models A$ (mai sbagliato). **Completezza:** $KB \models A \Rightarrow KB \vdash_i A$ (non perde nulla). La logica deve sempre essere corretta, non sempre completa.

Sintassi: simboli proposizionali, connettivi $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$. N simboli $\rightarrow 2^N$ modelli. $P \Rightarrow Q \equiv \neg P \vee Q$: falsa solo se P vera e Q falsa; **non è causale** (es. “Torino è in Lombardia $\Rightarrow$ Cesare governò Roma” è VERA perché l'antecedente è falso).

Verificare $KB \models P$: (1) model checking (2^N modelli, costoso) o (2) **theorem proving:** per **refutazione**, $KB \models P \iff (KB \wedge \neg P)$ è insoddisfacibile — si assume $\neg P$, si cerca una contraddizione.

CNF (congiunzione di clausole = disgiunzioni di letterali): elimina $\Leftrightarrow$, elimina $\Rightarrow$, De Morgan (nega verso l'interno), distribuisce $\vee$ su $\wedge$.

Risoluzione: da due clausole con letterali complementari, produce il resolvent (tutti i letterali tranne la coppia complementare, fattorizzato). **Modus ponens è un caso particolare** di risoluzione. Algoritmo **CP-RISOLUZIONE:** $CNF(KB \wedge \neg \text{query})$, risolvi iterativamente tutte le coppie finché non trovi la **clausola vuota** ($\rightarrow \text{true}$, $KB \models \text{query}$) o punto fisso ($\rightarrow \text{false}$). Corretta e completa.

Clausole di Horn: al più un letterale positivo (esattamente uno = clausola definita); catturano implicazioni con antecedente congiuntivo. Permettono inferenza in **tempo lineare** (base di Prolog).

- **Forward chaining:** parte dai fatti, applica modus ponens finché non deriva la query; **data-driven**, completo, lineare, ma può fare inferenze inutili.
- **Backward chaining:** parte dal goal, cerca regole che lo concludono, dimostra ricorsivamente le premesse; **goal-driven**, più efficiente/focalizzato, usato in Prolog.

6 Logica del primo ordine (FOL)

Limiti della proposizionale: non compatta (una regola per ogni individuo), non esprime relazioni fra oggetti. FOL: il mondo ha **oggetti in relazione**.

Sintassi: costanti (oggetti), predicati (proprietà/relazioni $\rightarrow$ vero/falso), funzioni ($\rightarrow$ un oggetto, non lo “costruiscono”, lo riferiscono), variabili, quantificatori $\forall \exists$, uguaglianza $=$.

Modello FOL $M = (D, I)$: D =dominio, I =interpretazione (costanti→oggetti, predicati→relazioni, funzioni→relazioni funzionali). Il valore di verità **dipende sempre dal modello**. FOL può avere **infiniti modelli** (dominio illimitato) → niente enumerazione come in proposizionale.

Quantificatori:

- $\forall x F$: vera se F vera per ogni x . **Regola pratica**: $\forall$ **va con** $\Rightarrow$ (per restringere a una sottoclasse: $\forall x \text{Partecipa}(x, C) \Rightarrow \text{Intelligente}(x)$).
- $\exists x F$: vera se F vera per almeno un x . **Regola pratica**: $\exists$ **va con** $\wedge (\exists x \text{Partecipa}(x, C) \wedge \text{Intelligente}(x))$. Usare $\forall$ con $\wedge$, o $\exists$ con $\Rightarrow$, produce quasi sempre formule sbagliate/banali ($\exists$ con $\Rightarrow$ è vera anche solo per un oggetto qualunque non nella sottoclasse).
- **Quantificatori diversi non commutano**: $\forall x \exists y \text{Ama}(x, y)$ (ognuno ama qualcuno, anche diverso) $\neq \exists y \forall x \text{Ama}(x, y)$ (esiste un unico y amato da tutti). Stesso tipo invece commuta liberamente.
- De Morgan generalizzato: $\forall x \neg F \equiv \neg \exists x F$, $\exists x \neg F \equiv \neg \forall x F$.

Uguaglianza: serve per dire “oggetti diversi” ($\neg(y = z)$), altrimenti FOL standard **non assume unicità dei nomi**. **Database semantics** (Prolog): Unique Names + Closed World + Domain Closure — molto più intuitiva ma va tenuta distinta dalla semantica standard FOL (R&N).

Inferenza: (1) **proposizionalizzazione** (UI: istanza $\forall$ con qualsiasi termine ground, KB equivalente; EI: istanza $\exists$ con una **costante nuova di Skolem**, una sola volta, equivalenza solo inferenziale) — completa per il **teorema di Herbrand**, ma **FOL è semidecidibile** (si conferma il sì, non sempre il no) ed è inefficiente (genera istanze inutili). (2) **Lifting** (più efficiente, in pratica usato sempre):

Modus Ponens Generalizzato (MPG): da $p'_1, \dots, p'_n$ e $(p_1 \wedge \dots \wedge p_n \Rightarrow q)$, se esiste θ con $p'_i \theta = p_i \theta \forall i$, si conclude $q\theta$. Richiede antecedente = congiunzione di letterali positivi.

Unificazione $\text{UNIFY}(F_1, F_2) = \theta$ tale che $F_1 \theta = F_2 \theta$ (idealmente il **Most General Unifier**). Può fallire per conflitto di variabile → **standardizzazione separata** (rinomina variabili fra formule diverse).

Clausole di Horn FOL (no funzioni = **DATALOG**): forward chaining (aggiunge fatto solo se non è **rinomina** di uno esistente; corretto, completo e termina su DATALOG, può non terminare con funzioni) e backward chaining (goal in stack, unifica con teste di clausole, corretto ma incompleto/depth-first, rischio loop).

Risoluzione in FOL (binary resolution + fattorizzazione, con unificazione): **refutation-complete**. Serve tradurre in **CNF**: elimina $\Leftrightarrow/\Rightarrow \rightarrow$ sposta negazioni dentro $\rightarrow$ standardizza variabili $\rightarrow$ **skolemizza** $\rightarrow$ elimina $\forall$ residui $\rightarrow$ distribuisce $\vee$ su $\wedge$.

Skolemizzazione: ogni $\exists y$ diventa una **funzione di Skolem** i cui argomenti sono tutte le variabili **universali nel cui scope** ricade (es. $\forall x \exists y F \rightarrow F[y/S(x)]$); se non c'è nessun $\forall$ esterno, diventa una **costante di Skolem** (=EI). Errore classico: skolemizzare con una costante fissa sotto un $\forall$ implica erroneamente che *tutti* condividano lo stesso valore.

7 Ontologie e Pianificazione

7.1 Tassonomie/Ontologie

Tassonomia = gerarchia (albero) di concetti via relazione **Is-a** (sottoclasse): le istanze ereditano le proprietà della superclasse (evita ridondanza). $\text{Disjoint}(S)$, $\text{ExhaustiveDec}(S, C)$, $\text{Partition}(S, C) = \text{Disjoint} \wedge \text{Exhaustive}$. **Part-of** (transitiva) descrive composizione strutturale; **Bunch-of** per aggregati senza relazioni interne.

T-box (schema/definizioni generali, intensionale) vs **A-box** (fatti su istanze, estensionale). Problemi: **eccezioni** alla norma (proprietà ereditate cancellabili in sottoclassi), **polisemia** (stessa parola, alberi tassonomici diversi — es. “cane”).

Ontologia generalizza la tassonomia: da albero (solo Is-a) a **grafo** (relazioni arbitrarie). Ogni tassonomia è un'ontologia, non viceversa.

Semantic Web (W3C): **RDF** (triple soggetto-predicato-oggetto, IRI = identificatori univoci, grafo RDF), **RDFS** (tassonomie su RDF), **OWL 2** (classi/proprietà/individui; `SubClassOf`, `EquivalentClasses`, `DisjointClasses`, `ObjectSomeValuesFrom/AllValuesFrom`; **open world assumption**, a differenza dei DB che sono closed-world), **SPARQL** (query), **FOAF** (ontologia sociale).

Relazioni fra ontologie: **Identical**, **Equivalent** (stesso contenuto, linguaggio diverso), **Extension** (una include l'altra), **Weakly-Translatable** (con perdita), **Strongly-Translatable** (fedele, senza perdita/inconsistenze), **Approx-Translatable** (con perdita e possibili inconsistenze).

Is-a/Part-of sono **statiche**: non bastano a rappresentare **azioni** → serve la pianificazione.

7.2 Pianificazione

Pianificare = costruire una sequenza di azioni da stato iniziale a goal. **PDDL** = standard per descriverla. Stato = congiunzione di atomi ground.

Situation Calculus (fondamento logico in FOL): **Azione** = funzione (non predicato) che restituisce un oggetto; **Situazione** = stato risultante da una sequenza di azioni; **Fluente** = proprietà che cambia con le azioni (ha sempre la situazione come parametro). $\text{Do}(\text{Azioni}, S)$ = situazione raggiunta. Due situazioni sono identiche solo se stessa storia (stato iniziale + sequenza azioni).

- **Assioma di applicabilità**: azione applicabile $\iff$ precondizioni vere.
- **Assioma di effetto**: azione applicabile $\Rightarrow$ effetti veri nella situazione risultante.

Frame problem: come rappresentare **ciò che un'azione NON modifica**? Non derivabile dai soli assiomi di effetto. Soluzioni: (1) enumerare esplicitamente un assioma di frame per ogni coppia azione $\times$ fluente non toccato (esplode combinatoriamente); (2) **assioma di stato successore** unico per fluente (fluente vero dopo $\iff$ l'azione lo rende vero, oppure era vero e l'azione non l'ha reso falso) — soluzione compatta standard.

Scomposizione in sottogoal: approccio base, ma i sottogoal possono **interagire**. **Anomalia di Sussman** (mondo dei blocchi: A su B, C isolato; goal A-B-C): risolvere un sottogoal (B su C) disfa i progressi dell'altro (A su B) e viceversa $\rightarrow$ serve **interleaving** dei passi dei due (sotto)piani, non esecuzione sequenziale a blocchi.

Situation Calculus storicamente fondamentale ma **poco usato in pratica** (mancano euristiche efficienti) — planner reali usano PDDL/planning graph/euristiche dedicate.

8 Machine Learning

8.1 Classificazione

Le **classi non esistono in natura**: sono definite dal progettista in base allo scopo; il sistema impara solo dai dati come sono etichettati.

Apprendimento supervisionato: training set (istanze (x, y) , x =attributi, y =classe) $\rightarrow$ **induzione** $\rightarrow$ modello classe = $f(\text{descrizione}) \rightarrow$ **deduzione** su nuove istanze; test set per **valutazione**.

Matrice di confusione (2 classi): Accuratezza = $(f_{11} + f_{22})/\text{tot}$, Error rate = $(f_{12} + f_{21})/\text{tot}$. **Matrice dei costi**: pesa diversamente i tipi di errore (es. falso negativo su malattia più grave di falso positivo). **Attenzione**: accuratezza alta su dataset **sbilanciato** non implica buon modello (classificatore banale “sempre classe maggioritaria” ottiene accuratezza alta senza aver imparato nulla).

Insidie pratiche: dataset non rappresentativo (bias di chi lo raccoglie), l'algoritmo non ha conoscenza di base umana (impara solo ciò che vede). Rischi etici in applicazioni ad alto impatto (riconoscimento facciale, mutui, guida autonoma) — **problem of the many hands** (responsabilità distribuita).

Rote learning: memorizza tutto, non generalizza; nuova istanza $\rightarrow$ cerca identica o simile (distanza), vota a maggioranza o pesata (pesi non persistenti, a differenza delle NN).

8.2 Alberi di decisione

Nodi interni = test su attributo; foglie = classe. **Algoritmo di Hunt** (ricorsivo): se D_t tutto della stessa classe $\rightarrow$ foglia; altrimenti scegli attributo di split, un figlio per valore, ricorri. Se istanze identiche ma classi diverse $\rightarrow$ foglia con classe di maggioranza. Strategia **greedy** (nessun backtracking).

Split per tipo di attributo: binario (2 figli), nominale (multivalore o binario raggruppato), ordinale (raggruppamento deve rispettare l'ordine), continuo (soglia $A \leq v$, o discretizzazione multi-soglia).

Misure di impurità: **Entropia** = $-\sum_i p(i|t) \log_2 p(i|t)$ (0=puro, max=equidistribuzione); **Gini** = $1 - \sum_i p(i|t)^2$; errore di classificazione. **Guadagno** = impurità(padre) - media pesata impurità(figli); con entropia si chiama **information gain**. **Attenzione**: information gain/Gini **favoriscono attributi con molti valori** (es. un ID univoco dà gain massimo ma è overfitting puro, zero generalizzazione) — mitigabile con split binari. **Rasoio di Occam**: a parità di prestazioni, preferire alberi più compatti.

8.3 Reti neurali

Perceptron (Rosenblatt): $\text{net} = \sum_i w_i X_i$, $Y = f(\text{net})$. Attivazione: gradino (non derivabile) o **sigmoide** (derivabile, necessaria per il gradiente). Geometricamente codifica un **iperpiano** nello spazio degli input: risolve solo problemi **linearmente separabili** (limite dimostrato con **XOR**). Apprendimento:

$$w_j \leftarrow w_j + \eta(d - o) x_j$$

(regola di correzione dell'errore); **Teorema di Novikoff**: converge solo se linearmente separabile. Epoca = intero learning set elaborato una volta.

MLP (Multi-Layer Perceptron): feed-forward, a strati (input=identità, hidden=sigmoide, output), fully-connected. Con ≥ 3 livelli e sigmoide, **approssimatore universale di funzioni**. Output: 1 neurone per 1-2 classi; **one-hot** (un neurone per classe) per multiclasse, preferito alla codifica binaria compatta perché più robusto/interpretabile.

Backpropagation: forward (calcola output) + backward (propaga l'errore, aggiorna pesi). **Discesa del gradiente**:

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}, \quad E = \frac{1}{2} \sum_i (t_i - y_i)^2.$$

Delta rule output: $\delta_j = y_j(1 - y_j)(t_j - y_j)$; hidden: $\delta_k = y_k(1 - y_k) \sum_j \delta_j w_{kj}$ (propaga all'indietro i δ pesati — da qui “backpropagation”, risolve il **credit assignment problem**).

Regressione: NN può approssimare funzioni continue (non solo classificare); errore valutato con **MSE** $= \frac{1}{n} \sum_i (y_o - y_d)^2$ invece di accuratezza/matrice di confusione.